

Set Covering

1 Introduction

We see here an important class of integer programming problems, namely set covering problems, and discuss two applications.

2 The set covering problem

A set covering problem (or set cover) is defined as follows.

Given:

- Points $V = \{i_1, \dots, i_n\}$;
- A collection of subsets $T_1, \dots, T_m$ of V .

Find: A *cover* of smallest cardinality, where $S \subseteq V$ is a cover if, for each $j = 1, 2, \dots, m$, we have $T_j \cap S \neq \emptyset$.

Example. We are given:

- $V = \{i_1, i_2, i_3, i_4, i_5\}$;
- $T_1 = \{i_1, i_2\}$, $T_2 = \{i_1, i_3, i_4\}$, $T_3 = \{i_2, i_3, i_5\}$, $T_4 = \{i_3, i_5\}$.

Then $\{i_1, i_4\}$ is not a cover (since T_4 is not covered), $\{i_1, i_2, i_5\}$ is a cover with 3 elements, and $\{i_2, i_3\}$ is a cover with 2 elements, thus a better one.

To formulate set cover as an integer program, we introduce a binary variable x_i for every $i \in V$, and write

$$\begin{aligned} \min \quad & \sum_{i \in V} x_i \\ & \sum_{i \in T_j} x_i \geq 1 \quad \text{for all } j = 1, \dots, m \quad (*) \\ & x \in \{0, 1\}^V \end{aligned}$$

The set covering problem can be generalized in many ways:

Non-unitary costs: Points $i_1, \dots, i_n$ may have different costs, and our goal is to find a cover so as to minimize the cost of the points it contains. In this case, if we let c_i be the cost of point i , we can replace the objective function of the IP above with

$$\sum_{i \in V} c_i x_i.$$

Multiple coverage: If each set T_j needs to be covered $d_j \in \mathbb{N}$ times, then we replace constraints $(*)$ in the IP above with

$$\sum_{i \in T_j} x_i \geq d_j \text{ for all } j = 1, \dots, m.$$

Multiple copies: If we can take multiple copies of every object, then we replace the binary constraints in the IP above with

$$x \in \mathbb{Z}_+^V.$$

Example. In the example above, introduce costs as follows:

- $c_1 = c_2 = 2, c_3 = 4, c_4 = c_5 = 1.$

Then, the cover $\{i_1, i_2, i_5\}$ is now preferred to the cover $\{i_2, i_3\}$ since the former has cost 5, while the latter has cost 6. If instead in the example above we let

- $d_1 = d_4 = 1, d_2 = d_3 = 2,$

Then none of $\{i_1, i_2, i_5\}$ and $\{i_2, i_3\}$ are feasible solutions (since they both contain only one element from set T_2 , while they should contain at least 2). A feasible solution is now $\{i_1, i_3, i_5\}$.

3 Hiring as a set covering problem

Suppose your company wants to hire one or more workers from a pool of n candidates. Each candidate has one or more skills from a set of m skills. We want to hire the minimum number of candidates so that every skill is possessed by one or more of the hired workers. See Figure 1 for an example.

This problem can be formulated as a set covering problem as follows.

- Items $i_1, \dots, i_n$ represent candidates;
- Sets $T_1, \dots, T_m$ represent workers; we have $i_\ell \in T_j$ if worker ℓ has the skill j .

Each of the generalizations of the set covering problem given above has a meaning in this context:

Non-unitary costs: Costs could represent salaries of workers, and we want to minimize the amount we spend on salaries.

Multiple coverage: We may want to have that some skills are possessed by more than one candidate (so that, e.g., when one candidate is on leave, there is some other worker who possesses that skill).

Multiple copies: There could be multiple candidates with exactly the same skills.

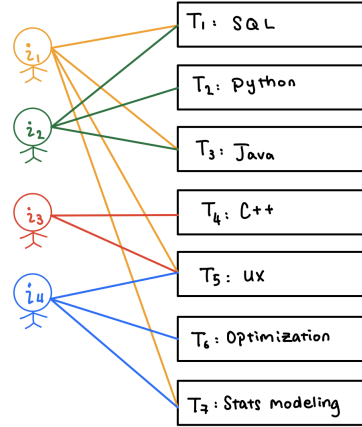


Figure 1: An application of set covering to hiring. Edges connect candidates with the skills they have.

4 A crew scheduling problem

Consider the following problem. Your shop is open from 8am to 6pm. Based on past customer data, you know that to serve your customers you will need a different number of clerks during the day:

hour	8-9	9-10	10-11	11-12	12-1	1-2	2-3	3-4	4-5	5-6
# clerks	4	5	5	3	6	11	9	7	5	4

Clerks can be hired for a consecutive shift of 3 hours. Find the minimum number of clerks to hire so that you will be able to serve your customers.

We formulate this problem as an instance of a generalization of the set covering problem as follows:

- We let $V = \{i_8, i_9, i_{10}, i_{11}, i_{12}, i_1, i_2, i_3\}$, where i_ℓ corresponds to the set of clerks who start working at time ℓ (note that it does not make sense to have clerks start at 4pm or later, since we have to pay them a 3 hours shift in any case).
- We let $T_8, T_9, \dots, T_5$ be the sets corresponding to every hour in our timetable; and we let $i_\ell \in T_j$ if clerks who start working at time ℓ will still be working at time j . So: $T_8 = \{8\}$, $T_9 = \{8, 9\}$, $T_{10} = \{8, 9, 10\}$, $T_{11} = \{9, 10, 11\}$, etc.
- Each set T_j needs to be covered as many times as indicated in the table above. So $d_1 = 4, d_2 = 5, d_3 = 5, \dots$

We can then formulate the problem as follows.

$$\begin{array}{rcccccccc}
\min & x_8 & +x_9 & +x_{10} & +x_{11} & +x_{12} & +x_1 & +x_2 & +x_3 & & \\
& x_8 & & & & & & & & \geq & 4 \\
& x_8 & +x_9 & & & & & & & \geq & 5 \\
& x_8 & +x_9 & +x_{10} & & & & & & \geq & 5 \\
& & x_9 & +x_{10} & +x_{11} & & & & & \geq & 3 \\
& & & x_{10} & +x_{11} & +x_{12} & & & & \geq & 6 \\
& & & & x_{11} & +x_{12} & +x_1 & & & \geq & 11 \\
& & & & & +x_{12} & +x_1 & +x_2 & & \geq & 9 \\
& & & & & & +x_1 & +x_2 & +x_3 & \geq & 7 \\
& & & & & & & +x_2 & +x_3 & \geq & 5 \\
& & & & & & & & x_3 & \geq & 4 \\
& x_8, & x_9, & x_{10}, & x_{11}, & x_{12}, & x_1, & x_2, & x_3 & \in & \mathbb{Z}_+
\end{array}$$